Sidi Mohamed Ben Abdellah University
National School of Applied Sciences - Fes, Morocco
Engineering, Systems and Applications Laboratory

Cours

Introduction à l'électronique Numérique

Filière : Deux années préparatoires (2AP) – ENSA Fès

MANSOURI Hanaa
hanaa.mansouri@usmba.ac.ma

2025 – 2026

Plan du cours

I. Systèmes de Numération et Codes

II. Logique Combinatoire

- A. Système combinatoire
- B. Logique booléenne
- C. Tableaux de Karnaugh

III. Synthèse de Systèmes Combinatoires

- A. Fonctions universelles
- B. Circuits combinatoires

IV. Logique Séquentielle

- A. Introduction
- B. Bascules RS
- C. Bascules JK
- D. Bascules D

I. Systèmes de Numération et Codes

Introduction

Le **système de numération** le plus adapté pour effectuer des calculs en informatique est le **système binaire**, ou **base 2**, qui ne comprend que deux symboles : 0 et 1.

Dans tout système numérique, les informations circulent sous forme de **mots binaires**, c'est-à-dire **des suites de 0 et de 1**.
Le **nombre d'éléments** composant ces mots est fixé à l'avance :
➞ par exemple, **un octet** est un mot binaire constitué de **huit éléments**.

La manière dont ces éléments sont écrits s'appelle le **code**.
Chaque élément d'un mot binaire est appelé **élément binaire (eb)** ou, plus couramment, **bit**, contraction du terme anglais *binary digit* (chiffre binaire)

Les **mots binaires** peuvent être écrits sous **différents codes**

selon le type d'information à traiter.

I. Systèmes de Numération et Codes

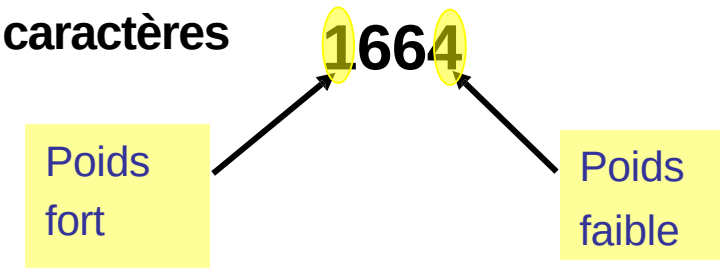
Définition

Si l'on considère la base décimale (10) :

- Les caractères définissant la base sont : 0,1,2,3,4,5,6,7,8,9

- Un mot (ou un chiffre) en base décimale est une combinaison de ces dix caractères

Ex: Un mot de 4 caractères



Ce nombre se décompose sous la forme :

$$1664 = 1 * 10^3 + 6 * 10^2 + 6 * 10^1 + 4 * \underline{10}^{0}$$

Poids

Base

I. Systèmes de Numération et Codes

Le codage binaire naturel (base 2)

Comme précédemment, tout nombre binaire peut être exprimé **comme un développement en puissances de 2**

$$1100101 = 1*2^6 + 1*2^5 + 0*2^4 + 0*2^3 + 1*2^2 + 0*2^1 + 1*2^0$$

Poids

Base

1100101

Poids fort (MSB)

Poids faible (LSB)

I. Systèmes de Numération et Codes

Le codage binaire naturel (base 2)

Code décimal	Code binaire naturel
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

I. Systèmes de Numération et Codes

Le codage hexadécimal (base 16)

- **Les caractères définissant la base sont : 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F**
- **Le nombre s'écrit :**

$$F2A_{Héx} = 15 * 16^2 + 2 * 16^1 + 10 * 16^0 = 8382_{Déc}$$

Le principal avantage de ce code est de pouvoir codé sur un mot court, un chiffre binaire important :

$$1001\ 0101\ 1101_b = 95D_H$$

1 caractère hexadécimale code 4 bits

I. Systèmes de Numération et Codes

Le codage binaire réfléchit ou code Grey (base 2)

Dans ce code, deux nombres successifs ne diffèrent que d'un bit. Il a été élaboré pour éviter les risques d'erreur de détection d'une information dans les systèmes réels.

Exemple : Prenons le nombre 5

- En binaire : 101
- Code de Gray :
  - Le 1er bit à gauche reste le même 1
  - Le 2e bit = 1 $\oplus$ 0 = 1
  - Le 3e bit = 0 $\oplus$ 1 = 1
  - ☞ Résultat : 111

Règle lorsque l'on compte :

(1) on change le bit de poids faible en premier. Si la combinaison existe, on passe au bit de rang supérieur

(2) On ne peut changer qu'un seul bit à la fois (d'une ligne sur l'autre)

I. Systèmes de Numération et Codes

Le code Grey

Code décimal	Code binaire naturel	Code binaire réfléchi
0	0	0000
1	1	0001
2	10	0011
3	11	0010
4	100	0110
5	101	0111
6	110	0101
7	111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

I. Systèmes de Numération et Codes

Le code ASCII

Sert à coder les chiffres, lettres et caractères spéciaux

Code ASCII étendu

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	`
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of xmit	EOT	CTRL-D	36	24	$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	2F	/	79	4F	O	111	6F	o
16	10	Data line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of xmit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	FS	CTRL-\	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	GS	CTRL-]	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	RS	CTRL-^	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	US	CTRL-_	63	3F	?	95	5F	_	127	7F	DEL

Exemples :
(= (28)Hex= (40)Dec
@ = (40)Hex=(64)Dec

I. Systèmes de Numération et Codes

Le code ASCII

Sert à coder les chiffres, lettres et caractères spéciaux

Code ASCII étendu

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	Ç	160	A0	á	192	C0	└	224	E0	α
129	81	ü	161	A1	í	193	C1	┴	225	E1	ß
130	82	é	162	A2	ó	194	C2	┬	226	E2	Γ
131	83	â	163	A3	ú	195	C3	├	227	E3	π
132	84	ä	164	A4	ñ	196	C4	─	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	┼	229	E5	σ
134	86	å	166	A6	ª	198	C6	╞	230	E6	µ
135	87	ç	167	A7	º	199	C7	╟	231	E7	τ
136	88	ê	168	A8	¿	200	C8	╚	232	E8	Φ
137	89	ë	169	A9	⌐	201	C9	╔	233	E9	Θ
138	8A	è	170	AA	¬	202	CA	╩	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	╦	235	EB	δ
140	8C	î	172	AC	¼	204	CC	╠	236	EC	∞
141	8D	ì	173	AD	¡	205	CD	═	237	ED	φ
142	8E	Ä	174	AE	«	206	CE	╬	238	EE	ε
143	8F	Å	175	AF	»	207	CF	╧	239	EF	∩
144	90	É	176	B0	░	208	D0	╨	240	F0	≡
145	91	æ	177	B1	▒	209	D1	╤	241	F1	±
146	92	Æ	178	B2	▓	210	D2	╥	242	F2	≥
147	93	ô	179	B3	│	211	D3	╙	243	F3	≤
148	94	ö	180	B4	┤	212	D4	Ö	244	F4	⌠
149	95	ò	181	B5	╡	213	D5	╒	245	F5	⌡
150	96	û	182	B6	╢	214	D6	╓	246	F6	÷
151	97	ù	183	B7	╖	215	D7	╫	247	F7	≈
152	98	ÿ	184	B8	╕	216	D8	╪	248	F8	°
153	99	Ö	185	B9	╣	217	D9	┘	249	F9	·
154	9A	Ü	186	BA	║	218	DA	┌	250	FA	·
155	9B	¢	187	BB	╗	219	DB	▀	251	FB	√
156	9C	£	188	BC	╝	220	DC	▄	252	FC	ⁿ
157	9D	¥	189	BD	╜	221	DD	▌	253	FD	²
158	9E	Pts	190	BE	╛	222	DE	▐	254	FE	■
159	9F	ƒ	191	BF	┐	223	DF	▀	255	FF	

I. Systèmes de Numération et Codes

Le code ASCII

Sert à coder les chiffres, lettres et caractères spéciaux

Code ASCII standard

ASCII	b_7 b_6 b_5	000	001	010	011	100	101	110	111
b_4 b_3 b_2 b_1	Hexa.	0	1	2	3	4	5	6	7
0000	0	(NUL)	(DLE)	espace	0	@	P	`	p
0001	1	☺ (SOH)	(DC1)	!	1	A	Q	a	q
0010	2	(STX)	↕ (DC2)	"	2	B	R	b	r
0011	3	♥ (ETX)	!! (DC3)	#	3	C	S	c	s
0100	4	♦ (EOT)	¶ (DC4)	¤	4	D	T	d	t
0101	5	♣ (ENQ)	_ (NAK)	%	5	E	U	e	u
0110	6	♠ (ACK)	(SYN)	&	6	F	V	f	v
0111	7	• (BEL)	↕ (ETB)	‘	7	G	W	g	w
1000	8	(BS)	↑ (CAN)	(	8	H	X	h	x
1001	9	○ (HT)	↓ (EM)	)	9	I	Y	i	y
1010	A	(LF)	→ (SUB)	*	:	J	Z	j	z
1011	B	(VT)	← (ESC)	+	;	K	[	k	{
1100	C	(FF)	(FS)	,	<	L	\	l	|
1101	D	(CR)	↔ (GS)	-	=	M	]	m	}
1110	E	(SO)	(RS)	.	>	N	^	n	¯
1111	F	☼ (SI)	(US)	/	?	O	_	o	DEL

I. Systèmes de Numération et Codes

Conversions

1) Conversion d'un nombre écrit en base quelconque vers le décimal

l'équivalent décimal d'un nombre N écrit dans une base b quelconque s'obtient par application directe de la forme polynomiale: $N = \sum_{i=0}^{n} a_i b^i$, Où $a_i \in \{0,1,2,3,4,5,6,7,8,9\}$

Exemple:

$(10111)_2 = 1\times2^4+0\times2^3+1\times2^2+1\times2^1+1\times2^0 = 1\times16+0\times8+1\times4+1\times2+1\times1 = 16+0+4+2+1 = (23)_{10}$

$(2145)_8 = 2\times8^3+1\times8^2+4\times8^1+5\times8^0 = 2\times512+1\times64+4\times8+5\times1 = 1125$

2) Conversion d'un nombre décimal en binaire

Il s'agit d'une répétition de divisions par 2 jusqu'à ce que le quotient devienne nul. Les restes obtenus lors de chaque division correspondent aux bits a_i à déterminer. On place le premier reste à la position du LSB (bit de poids faible) et le dernier reste à la position du MSB (bit de poids fort).

Exemple: Convertir le nombre décimal 13 en binaire

13 ÷ 2 = 6 reste **1**

6 ÷ 2 = 3 reste **0** $\Rightarrow (13)_{10} = (1101)_2$

3 ÷ 2 = 1 reste **1**

1 ÷ 2 = 0 reste **1**

I. Systèmes de Numération et Codes

Conversions

3) Conversion d'un nombre décimal en octal

Même principe qu'avant sauf qu'au lieu de diviser par 2, on divise par 8.

4) Conversion d'un nombre décimal en hexadécimal

Même principe qu'avant sauf qu'au lieu de diviser par 8, on divise par 16.

5) Conversion binaire vers octal

On passe du binaire à l'octal en groupant les bites par blocs de 3 en allant vers la gauche puis on fait correspondre à chaque bloc son équivalent décimal.

Exemple:

$(100\ 111\ 010\ 001)_2 = (4\ 7\ 2\ 1)_8$

$(10\ 111)_2 = (2\ 7)_8$

6) Conversion octal vers binaire

pour cela il suffit d'associer à chaque chiffre du nombre octal son équivalent binaire sur 3 bits.

Exemple: $(402)_8 = (100\ 000\ 010)_2$

I. Systèmes de Numération et Codes

Conversions

7) Conversion binaire vers hexadécimal

On passe du binaire à l'hexadécimal en groupant les bits par blocs de 4 en allant vers la gauche puis on fait correspondre à chaque bloc son équivalent hexadécimal.

Exemple: $(1010\ 1100\ 1101)_2 = (ACD)_{Héx}$

8) Conversion hexadécimale vers binaire

Cela consiste à associer à chaque caractère du nombre hexadécimale son équivalent binaire sur 4 bits.

Exemple: $(\mathbf{1}FA2)_{Héx} = (\mathbf{0001}1111\mathbf{1010}0010)_2$

9) Conversion en complément à 1

On passe facilement d'un nombre binaire en son complément à 1 en inversant tous les bits (0 deviennent des 1 et les 1 des 0).

Exemple:

Nombre binaire	Complément à 1
$(0111)_2$	$(1000)_2$

I. Systèmes de Numération et Codes

Conversions

10) Conversion en complément à 2

Le complément à 2 d'un nombre binaire est tout simplement son complément à 1 auquel on additionne un 1.

Exemple:

Nombre binaire	Complément à 2
$(0111)_2$	$(1001)_2$

Le codage des nombres signés

Note : Le bit de poids fort est utiliser pour coder le signe :

→ 0 : si l'entier est positif

→ 1 : si l'entier est négatif

Les nombres signés sont représentés selon trois notations :

-Notation module plus signe,

Bit de signe	Module

-Notation complément à 1,

Bit de signe	Complément à 1

-Notation complément à 2.

Bit de signe	Complément à 2

I. Systèmes de Numération et Codes

Le codage des nombres signés

Exemples :

- Notation module plus signe,

Bit de signe	Module

+12 → 0 1100

-12 → 1 1100

- Notation complément à 1,

Bit de signe	Complément à 1

+12 → 0 1100

-12 → 1 0011

- Notation complément à 2.

Bit de signe	Complément à 2

+12 → 0 1100

-12 → 1 0100

Électronique combinatoire

II. Logique Combinatoire

A. Système combinatoire

On appelle **système combinatoire** tout système numérique dont les sorties sont exclusivement définies à partir des variables d'entrée (figure 1).

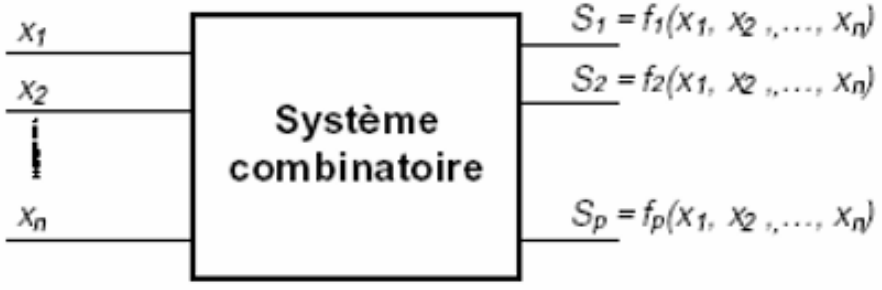


Figure 1 : système combinatoire générant p sorties S_i à partir des n variables d'entrée x_i.

Pour réaliser de tels systèmes, on fait appel à des portes dites *logiques*, correspondant aux fonctions définies par l'algèbre de Boole.

II. Logique combinatoire

B. La logique Booléenne

Variable binaire

On appelle **variable binaire** (ou logique) une variable qui prend ses valeurs dans l'ensemble $\{0, 1\}$.

Exemple : état d'un interrupteur, d'un bouton poussoir, etc….

Soit ***a*** la variable associée à l'état d'un bouton poussoir, alors $a = 0$ (faux ou bas) signifie qu'il n'est pas actionné, $a = 1$ (vrai ou haut) signifie qu'il est actionné.

Equation logique

On appelle **équation logique** une combinaison de plusieurs variables logiques donnant l'état d'une variable dite de sortie associée. Cette combinaison est réalisée à l'aide d'opérations logiques :

Soit xi ($i \in [1, n]$) les variables d'entrée. L'équation $A = f(xi)$ définit l'état de la variable de sortie A.

Table de vérité

La **table de vérité** représente l'état de la variable de sortie pour chacune des combinaisons des n variables d'entrée ($2n$ lignes).

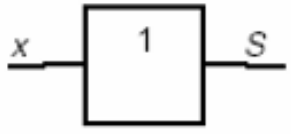
I. Les Opérateurs

1. Opérateur OUI (YES)

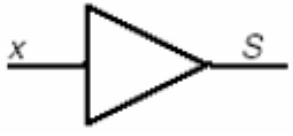
L'opération (ou opérateur) OUI est dite **unaire** (ne s'applique qu'à une seule opérande). Elle affecte à la variable de sortie l'état logique de la variable d'entrée.

Equation: *x* est la l'entrée, *s* la sortie avec *s* = *x*.

Symbole (norme IEC)



Symbole (norme IEEE)



x	*S*
0	0
1	1

Table de vérité

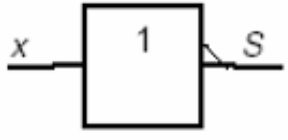
I. Les Opérateurs

2. Opérateur NON ou inverseur (not- INV)

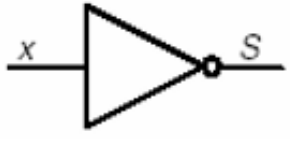
L'opération (ou opérateur) NON est la fonction unaire qui affecte à la variation de sortie de l'état complémentaire (l'état opposé) de la variation d'entrée.

Equation: x et l'entrée, s la sortie avec s = $\bar{x}$ (prononcer x barre).

Symbole (norme IEC)



Symbole (norme IEEE)



x	S
0	1
1	0

Table de vérité

II. Logique combinatoire

B. La logique Booléenne

I. Les Opérateurs

3. Opérateur ET (AND)

L'opération ET est appelée *produit logique*. Son symbole est celui de la multiplication (le point ·), mais on le lit "et". C'est un opérateur binaire qui donne à la variable de sortie la valeur 1 si, et seulement si, toutes les variables d'entrée valent 1 simultanément.

Equation : x et y les entrées, S la sortie, $S = x.y = xy$.

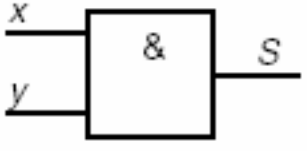
On note aussi l'opération ET par un V retourné :

$x.y = x \wedge y$ (penser à l'intersection d'ensembles).

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Table de vérité

Symbole (norme IEC)



Symbole (norme IEEE)



II. Logique combinatoire

B. La logique Booléenne

I. Les Opérateurs

4. Opérateur OU (OR)

L'opération OU est appelée *somme logique*. Son symbole est celui de l'addition (+), mais on le lit "ou". C'est un opérateur binaire qui donne à la variable de sortie la valeur 1 si au moins une des variables d'entrée vaut 1.

Equation : x et y les entrées, S la sortie, S=x + y

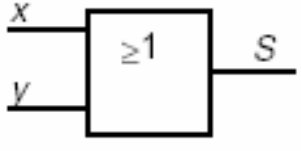
On note aussi l'opération OU par un V :

x+y $= x \vee y$ (penser à l'union d'ensembles).

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

Table de vérité

Symbole (norme IEC)



Symbole (norme IEEE)



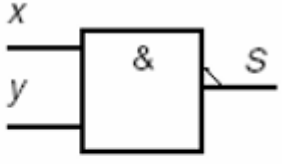
I. Les Opérateurs

5. Opérateur NON-ET (NAND)

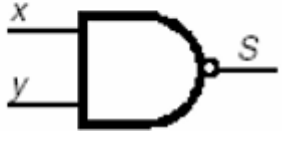
Cette fonction logique est le résultat de l'association d'un NON et d'un ET. C'est un opérateur binaire qui donne à la variable de sortie la valeur 0 si, et seulement si, les deux variables d'entrée sont égales à 1 simultanément.

Équation : x et y sont les entrées, s est la sortie, avec $s = \overline{x \cdot y}$
On note aussi l'opérateur NAND par une flèche montante: $s = \overline{x \cdot y} = x \uparrow y$

Symbole (norme IEC)



Symbole (norme IEEE)



x	y	$x \uparrow y$
0	0	1
0	1	1
1	0	1
1	1	0

Table de vérité

II. Logique combinatoire

B. La logique Booléenne

I. Les Opérateurs

6. Opérateur NON-OU (NOR)

Cette fonction logique est le résultat de l'association d'un NON et d'un OU. C'est un opérateur binaire qui donne à la variable de sortie la valeur 1 si, et seulement si, toutes les variables d'entrée sont égales à 0 simultanément.

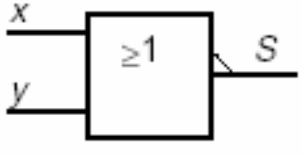
Equation : x et y les entrées, S la sortie, $S = \overline{x+y}$.

On note aussi l'opération NOR par une flèche descendante : $S = \overline{x+y} = x \downarrow y$

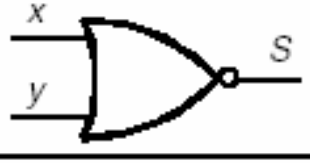
x	y	$x \downarrow y$
0	0	1
0	1	0
1	0	0
1	1	0

Table de vérité

Symbole (norme IEC)



Symbole (norme IEEE)



II. Logique combinatoire

B. La logique Booléenne

I. Les Opérateurs

7. Opérateur OU EXCLUSIF (XOR)

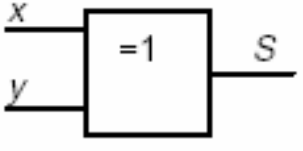
Cet opérateur logique binaire donne la valeur 1 uniquement lorsqu'une seule des entrées est égale à 1.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Table de vérité

Equation : x et y les entrées, S la sortie, $S = x \oplus y$.

Symbole (norme IEC)



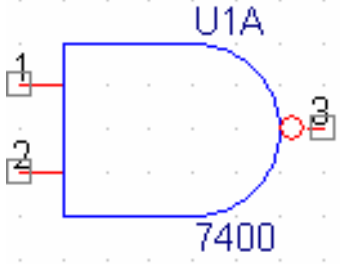
Symbole (norme IEEE)



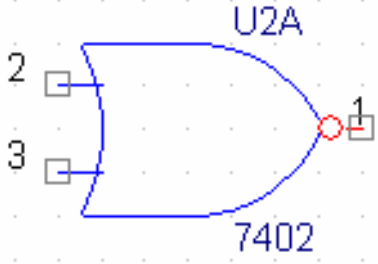
II. Logique combinatoire

B. La logique Booléenne

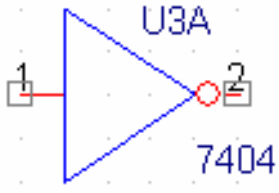
Toutes les portes élémentaires logiques sont associées à un composants codé



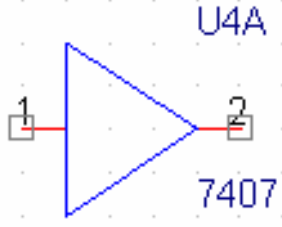
NON-ET(NAND)



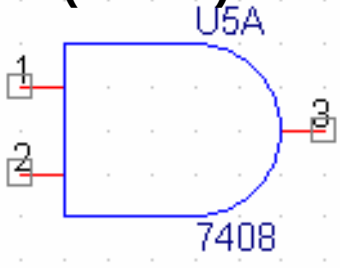
NON-OU(NOR)



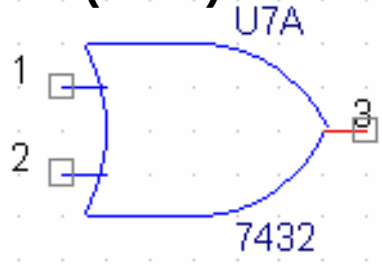
NON(NO)



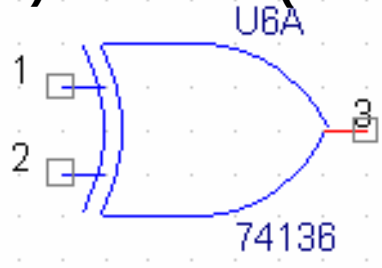
OUI(YES)



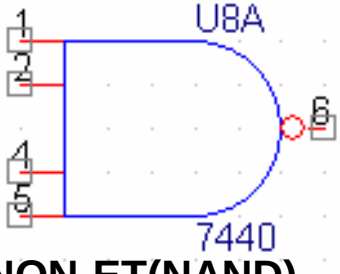
ET(AND)



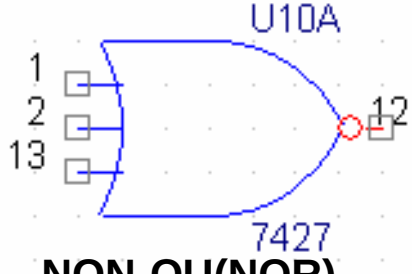
OU(OR)



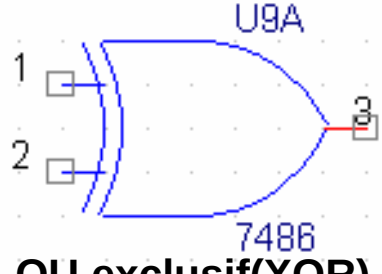
OU exclusif(XOR)



NON-ET(NAND)



NON-OU(NOR)



OU exclusif(XOR)

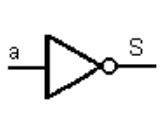
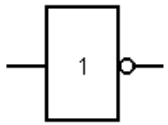
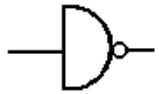
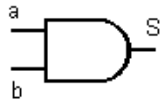
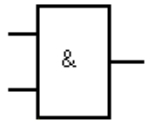
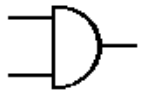
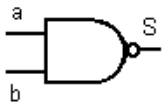
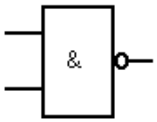
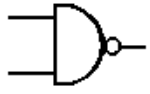
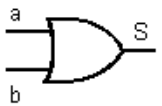
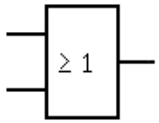
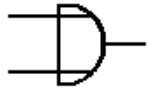
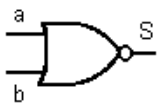
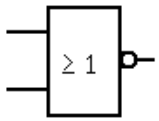
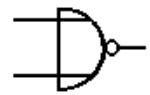
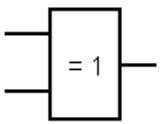
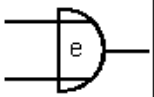
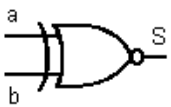
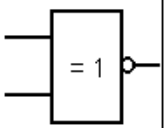
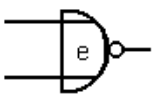
FONCTION	EQUATION	SYMBOLES			TABLES DE VERITE
		International	Français	Allemand	
NON	$S = \overline{a}$				a | S 0 | 1 1 | 0
ET	$S = a \cdot b$				a | b | S 0 | 0 | 0 0 | 1 | 0 1 | 0 | 0 1 | 1 | 1
NAND	$S = \overline{a \cdot b}$				a | b | S 0 | 0 | 1 0 | 1 | 1 1 | 0 | 1 1 | 1 | 0
OU	$S = a + b$				a | b | S 0 | 0 | 0 0 | 1 | 1 1 | 0 | 1 1 | 1 | 1
NOR	$S = \overline{a + b}$				a | b | S 0 | 0 | 1 0 | 1 | 0 1 | 0 | 0 1 | 1 | 0
OU Exclusif	$S = a \oplus b$				a | b | S 0 | 0 | 0 0 | 1 | 1 1 | 0 | 1 1 | 1 | 0
NOR Exclusif	$S = \overline{a \oplus b}$				a | b | S 0 | 0 | 1 0 | 1 | 0 1 | 0 | 0 1 | 1 | 1

Fig. 62. - Symboles graphiques des différentes fonctions logiques.

II. Logique combinatoire

B. La logique Booléenne

II. Les expressions logiques et leurs simplifications

1. Propriétés

- Commutativité

$a + b = b + a$ (commutativité de l'opération OU)

$a.b = b.a$ (commutativité de l'opération ET)

- Associativité

$a + (b + c) = (a + b) + c = a + b + c$ (associativité de l'opération OU)

$(ab)c = a(bc) = abc$ (associativité de l'opération ET)

- Distributivité

$(a + b).c = ac + bc$ (distributivité du produit logique sur la somme logique)

$ab + c = (a + c).(b + c)$ (distributivité de la somme logique sur le produit logique)

Les parenthèses imposent une priorité supérieure.

II. Logique combinatoire

B. La logique Booléenne

II. Les expressions logiques et leurs simplifications

2. Les autres propriétés

$a + 0 = a$	Element neutre	$a \cdot 1 = a$
$a + 1 = 1$	Element absorbant	$a \cdot 0 = 0$
$a + a = a$	Idem potence (redondance)	$a \cdot a = a$
$a + \bar{a} = 1$	Propriété du complément	$a \cdot \bar{a} = 0$
$a + b = b + a$	Commutativité	$a \cdot b = b \cdot a$
$a + b + c = a + (b + c) = (a + b) + c$	Associativité	$a\,b\,c = a\,(b\,c) = (a\,b)\,c$

3. Les propriétés déduites

Propriété d'absorbtion : Lorsqu'une somme logique contient un terme et un de ses multiple n peut négliger le multiple. Exemple : $x + x\,y = x$

Règle du multiple du complément $a + \bar{a}\,b = (a + \bar{a})(a + b) = a + b$

II. Logique combinatoire

B. La logique Booléenne

II. Les expressions logiques et leurs simplifications

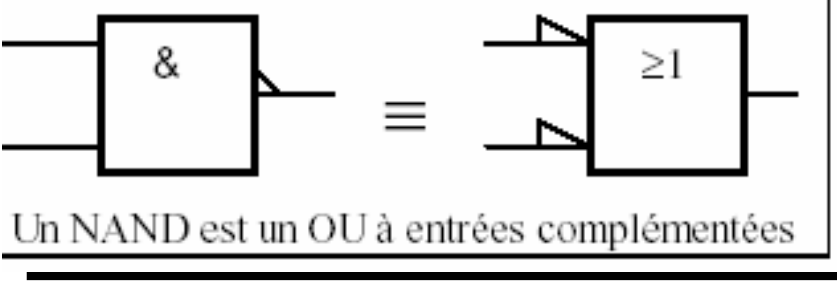
4. Propriétés de De Morgan

But: Exprimer les opérateurs **ET**, **OU** et **NON** exclusivement à l'aide d'opérateurs **NOR** seuls ou **NAND** seuls. On dit que les opérateurs **NOR** et **NAND** sont **universels** ou **complets**.

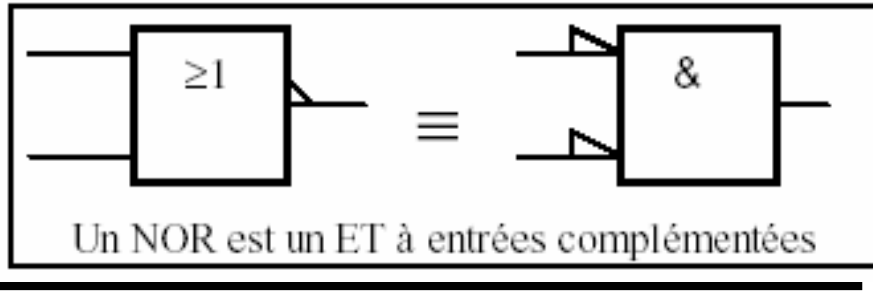
Premier théorème : $\overline{a+b} = \overline{a} \cdot \overline{b}$ généralisation $\overline{\sum_{i=1}^{n} a_i} = \prod_{i=1}^{n} \overline{a_i}$ où les les a_i sont les variables, $i \in [1,n]$.

Second théorème : $\overline{a \cdot b} = \overline{a} + \overline{b}$ généralisation $\overline{\prod_{i=1}^{n} a_i} = \sum_{i=1}^{n} \overline{a_i}$ où les les a_i sont les variables, $i \in [1,n]$.

Signification pratique



Un NAND est un OU à entrées complémentées



Un NOR est un ET à entrées complémentées

B. La logique Booléenne

On considère l'exemple suivant :

$$l = \left(a + \bar{b} + c\right) \cdot \left(a + \bar{c}\right) \cdot \left(\bar{a} + \bar{b}\right)$$

Résoudre par l'algèbre en développant.

Résoudre par l'utilisation des propriétés de De Morgan

II. Logique combinatoire

B. La logique Booléenne

On considère l'exemple suivant : $l = (a+\overline{b}+c)\cdot(a+\overline{c})\cdot(\overline{a}+\overline{b})$

Voici une simplification propre (algèbre booléenne) :

1. Développer les deux derniers facteurs :

$$(a+\bar{c})(\bar{a}+\bar{b}) = a\bar{b} + \bar{a}\,\bar{c} + \bar{b}\,\bar{c}$$

2. Multiplier par $(a+\bar{b}+c)$ et absorber :

$$(a+\bar{b}+c)(a\bar{b}+\bar{a}\bar{c}+\bar{b}\bar{c})$$

$$= (a\bar{b}+a\bar{b}+a\bar{b}c) \;+\; (0+\bar{a}\bar{c}\bar{b}+0) \;+\; (\bar{b}\bar{c}a+\bar{b}\bar{c}+0)$$

$$= (a\bar{b}+a\bar{b}) \;+\; (\bar{a}\bar{c}\bar{b}) \;+\; (\bar{b}\bar{c})$$

$$= a\bar{b}+\bar{a}\bar{c}\bar{b}+\bar{b}\bar{c}$$

$$= a\bar{b}+\bar{b}\bar{c}$$

$$= \bar{b}(a+\bar{c})$$

II. Logique combinatoire

B. La logique Booléenne

Résolution par l'utilisation des propriétés de De Morgan

On considère l'expression : $l = \left(a+\overline{b}+c\right)\cdot\left(a+\overline{c}\right)\cdot\left(\overline{a}+\overline{b}\right)$

Rappel : Tu n'es pas obligé de commencer par Morgan 1; chaque loi s'applique selon la forme que tu rencontres. On applique De Morgan à l'endroit où il existe une négation globale. Ensuite, si à l'intérieur il y a encore des groupes avec des négations, tu peux réappliquer De Morgan autant de fois que nécessaire.

- Si la barre couvre un OU → appliquer Morgan 1 $\overline{a+b} = \overline{a}\cdot\overline{b}$
- Si la barre couvre un ET → appliquer Morgan 2 : $\overline{a\cdot b} = \overline{a}+\overline{b}$

1) Application de la loi de De Morgan 1 sur chaque parenthèse

Noter que : $A + B = \overline{\overline{A+B}} = \overline{\overline{A}\cdot\overline{B}}$

Donc : $a+\overline{b}+c = \overline{\overline{a+\overline{b}+c}} = \overline{\overline{a}\cdot b\cdot\overline{c}}$

$a+\overline{c} = \overline{\overline{a+\overline{c}}} = \overline{\overline{a}\cdot c}$

$\overline{a}+\overline{b} = \overline{\overline{\overline{a}+\overline{b}}} = \overline{a\cdot b}$

Donc : $l = \overline{\overline{a}\,b\,\overline{c}}\cdot\overline{\overline{a}\,c}\cdot\overline{ab}$

2) Utilisation de la loi de De Morgan 1 sur l'ensemble de la fonction *l*

Noter que : $\overline{A}\cdot\overline{B} = \overline{A+B}$

Donc : $l = \overline{\overline{a}\,b\,\overline{c}}\cdot\overline{\overline{a}\,c}\cdot\overline{ab} = \overline{(\overline{a}b\overline{c})+(\overline{a}c)+(ab)}$

B. La logique Booléenne

3) Simplification algébrique de la fonction l à l'aide des propriétés de l'algèbre de Boole

On regroupe les termes :

$$l = \overline{\overline{a}(b\overline{c} + c) + ab}$$

Or :

$$b\overline{c} + c = b + c$$

Donc :

$$l = \overline{\overline{a}(b + c) + ab}$$

Factorisation :

$$l = \overline{\overline{a}b + ab + \overline{a}c}$$

$$l = \overline{b(\overline{a} + a) + \overline{a}c}$$

Or :

$$\overline{a} + a = 1$$

Donc :

$$l = \overline{b + \overline{a}c}$$

Ou sous forme produit :

$$\boxed{l = \overline{b}\,(a + \overline{c})}$$

II. Logique combinatoire

B. La logique Booléenne

II. Les expressions logiques et leurs simplifications

5. Formes canonique d'une fonction logique.

Toute fonction logique peut être représentée par sa première forme canonique (somme de produit) ou sa deuxième forme canonique (produit de somme).

5.1 Détermination de la première forme canonique:

Pour trouver l'expression d'une fonction logique sous sa première forme canonique, on part de sa table de vérité ou de sa table de karnaugh et on développe suivant les "1" de la fonction; en respectant le principe suivant: un "1" correspond à la variable non complémentée et un "0" correspond à la variable complémentée

Exemple 1: Expression de la fonction ou exclusif $\mathbf{S = a \oplus b}$

a	b	S
0	0	0
0	1	1
1	0	1
1	1	0

$$S = \bar{a} \cdot b + a \cdot \bar{b}$$

Réalisation:

- à l'aide des opérateurs NON, ET, OU

$$S = \bar{a} \cdot b + a \cdot \bar{b}$$

- à l'aide de l'opérateurs NAND

$$S = \overline{\overline{(a \cdot \overline{(a \cdot b)})} \cdot \overline{(b \cdot \overline{(a \cdot b)})}}$$

II. Logique combinatoire

B. La logique Booléenne

II. Les expressions logiques et leurs simplifications

5. Formes canonique d'une fonction logique.

5.1 Détermination de la deuxième forme canonique:

Pour trouver l'expression d'une fonction logique sous sa deuxième forme canonique, on part de sa table de vérité ou sa table de karnaugh et on développe suivant les "0" de la fonction ; en respectant le principe suivant : un "1" correspond à la variable complémentée et un "0" correspond à la variable non complémentée.

Exemple 1: Expression de la fonction ou exclusif **S = a ⊕ b**

a	b	S
0	0	0
0	1	1
1	0	1
1	1	0

$$S = (a + b) \cdot (\bar{a} + \bar{b}) = a \cdot \bar{b} + b \cdot \bar{a}$$

Réalisation:

- à l'aide des opérateurs NON, ET, OU

  $S = a \cdot \bar{b} + b \cdot \bar{a}$

- à l'aide des opérateurs NOR :

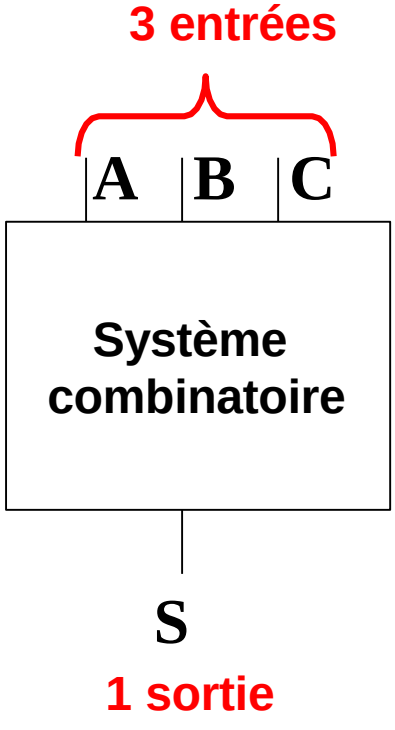
  En utilisant De Morgan : $S = \overline{\overline{a+b} + \overline{a+a} + \overline{b+b}}$

II. Logique combinatoire

B. La logique Booléenne

III. Les tables de vérités et leurs utilisations

Une table de vérité présente l'ensemble des états possibles de la sortie pour **toutes** les combinaisons des variables d'entrée.



Les entrées — La sortie

MSB — LSB

C	B	A	S
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**0**
1	0	0	**1**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

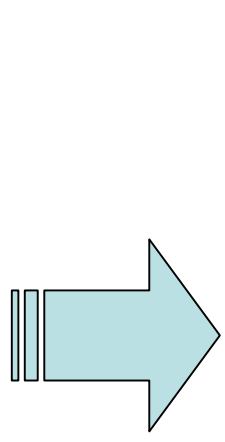
II. Logique combinatoire

C. Les tableaux de Karnaugh

I. Définition

- Le tableau de Karnaugh est une représentation graphique des combinaisons logiques d'un système.
- Il comporte 2^n cases, où *n* est le nombre de variables, organisées selon le code Gray. Chaque case correspond à une combinaison d'entrées et contient la valeur de sortie correspondante.

C	B	A	**S**
0	0	0	**0**
0	0	1	**1**
0	1	0	**0**
0	1	1	**0**
1	0	0	**1**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**



Attention au codage : Code GRAY

c \ ba	00	01	11	10
0	0	1	0	0
1	1	1	1	1

C. Les tableaux de Karnaugh

II. Utilisation

- Les tableaux de Karnaugh servent à **simplifier les équations logiques**.

- Les cases peuvent être **regroupées par 2, 4, 8, …, 2^n** pour éliminer les variables qui changent d'état :

2 cases → élimine 1 variable

2^x cases → élimine *x* variables

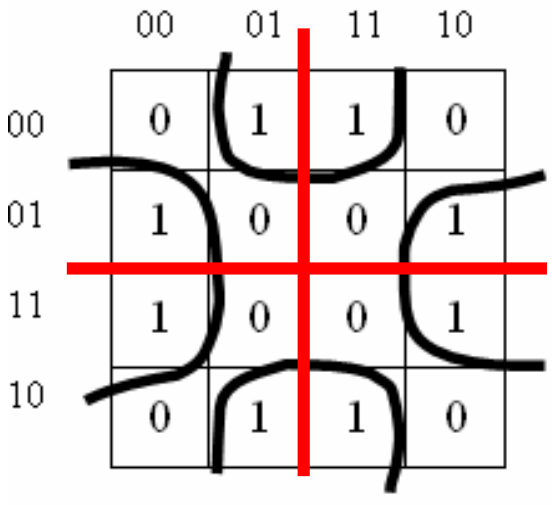


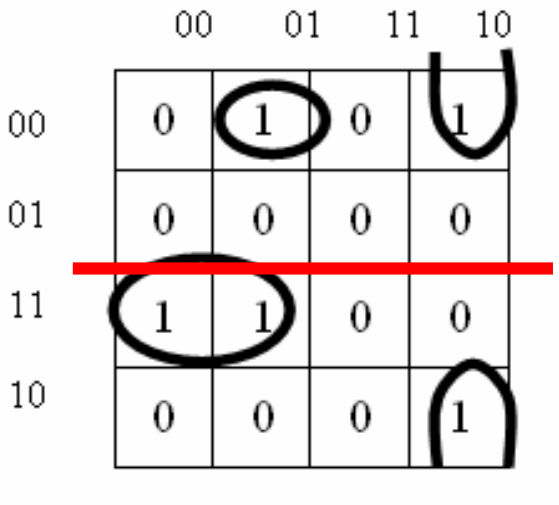
$$S = \overline{B}A + C$$

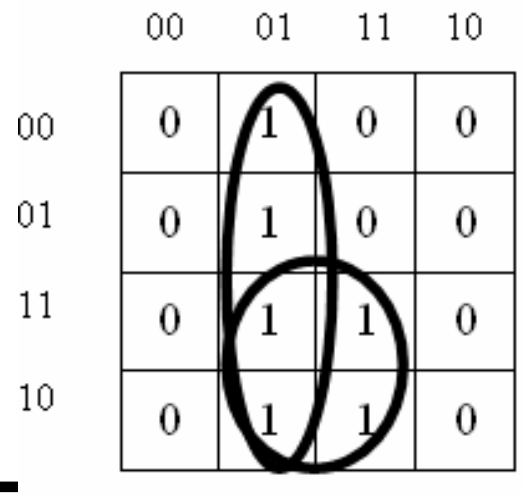
II. Logique combinatoire

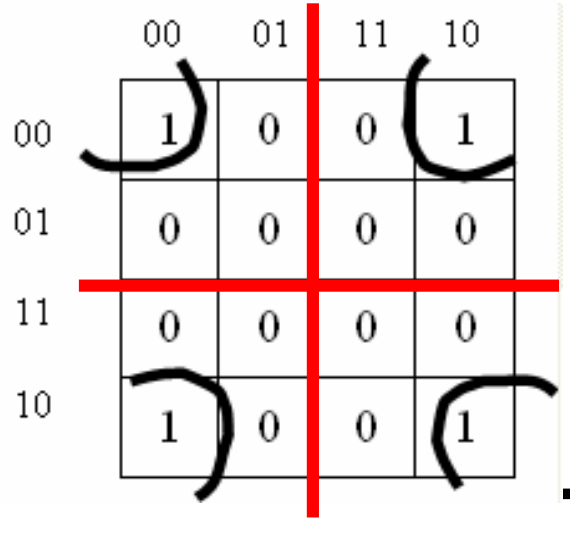
C. Les tableaux de Karnaugh

III. Des exemples de regroupements autorisés





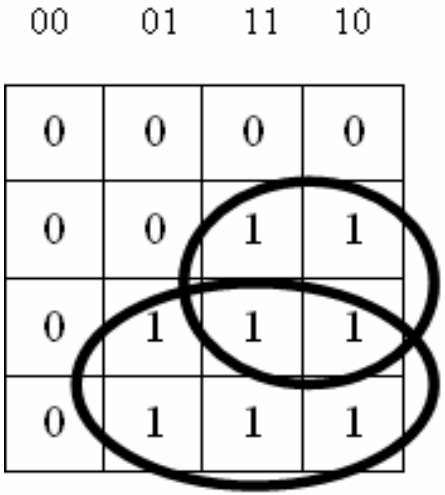


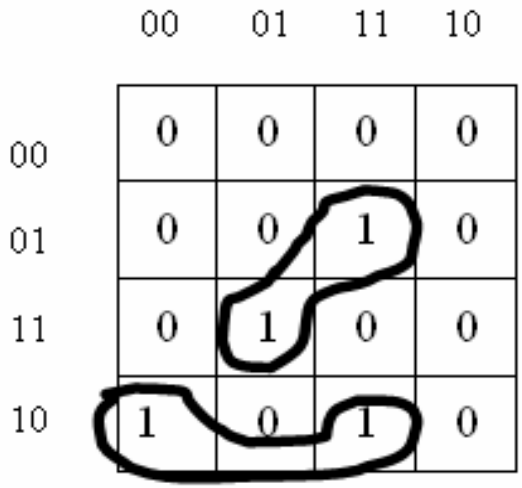


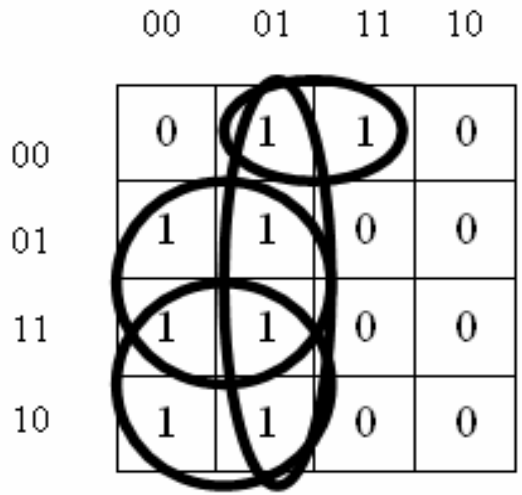
II. Logique combinatoire

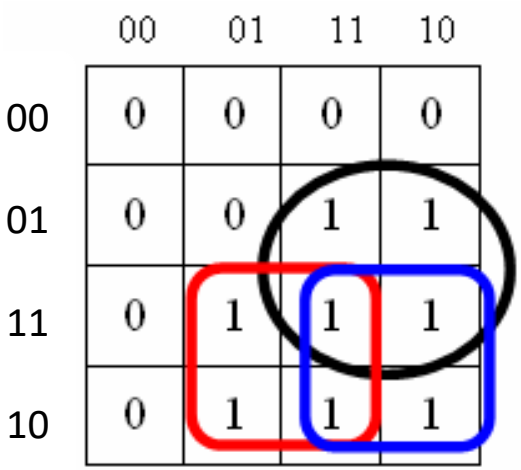
C. Les tableaux de Karnaugh

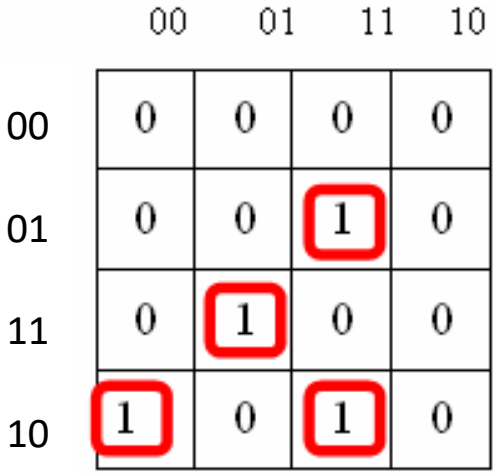
IV. Des exemples de regroupements non autorisés ou redondants

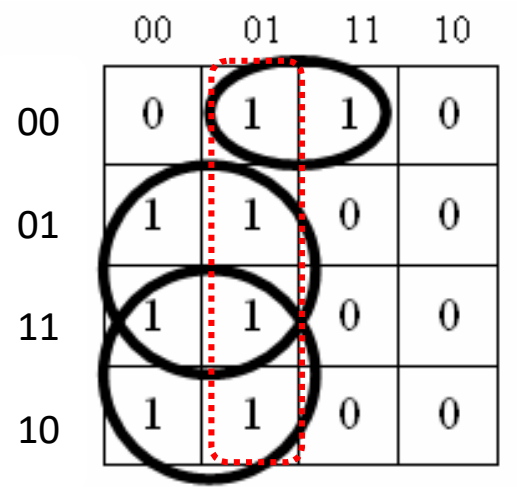












La correction

Applications de l'électronique combinatoire

III. Synthèse de Systèmes Combinatoires

But : réaliser par un assemblage de portes logiques

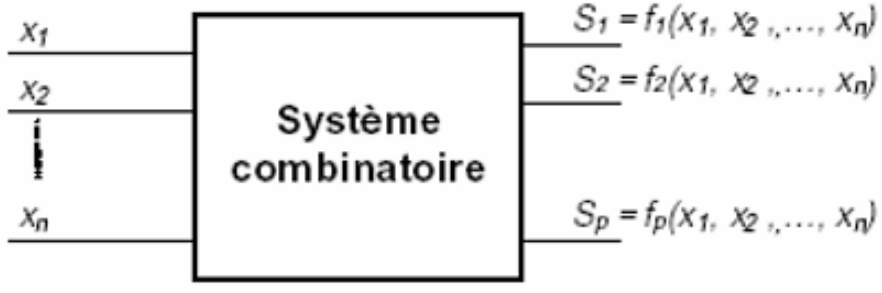


Figure 1 : système combinatoire générant p sorties S_i à partir des n variables d'entrée x_i.

1. À partir du cahier des charges, identifier les entrées et les sorties du système.
2. Établir la table de vérité qui décrit le fonctionnement du système.
3. Déterminer les équations logiques simplifiées de chaque sortie en fonction des entrées.
4. Réaliser le schéma logique en assemblant les portes tout en respectant les contraintes du cahier des charges.

III. Synthèse de Systèmes Combinatoires

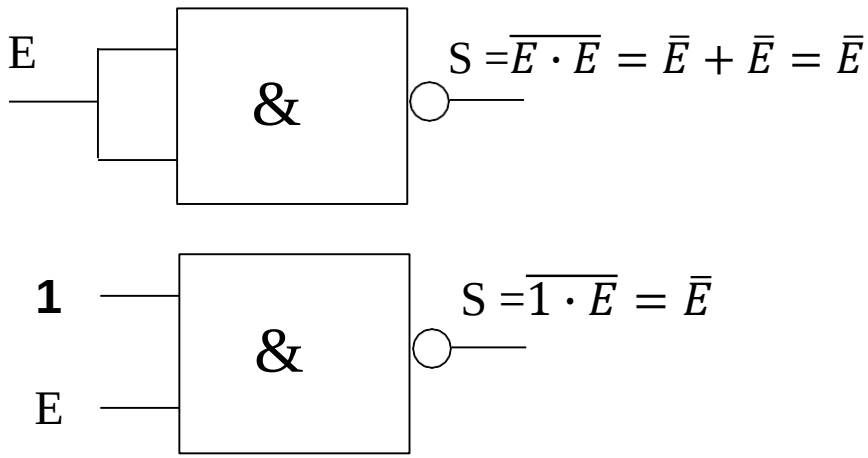
A. Les fonctions universelles

Dans la pratique, les **fonctions logiques les plus utilisées** ne sont pas les trois fonctions de base (**NON**, **OU**, **ET**), mais plutôt les **fonctions NAND** et **NOR**.
En effet, pour des **raisons technologiques** que nous verrons plus loin, il est possible de **réaliser les trois fonctions de base** en utilisant **uniquement** des portes **NAND** ou **NOR**.
Ainsi, on peut construire **n'importe quelle fonction logique combinatoire** à partir de ces portes.
☞ C'est pourquoi les **portes NAND** et **NOR** sont appelées **éléments de connexion universels**.

A.1) Fonction NON (Inverseur)

E_1	E_2	S
0	0	1
0	1	1
1	0	1
1	1	0



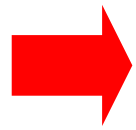


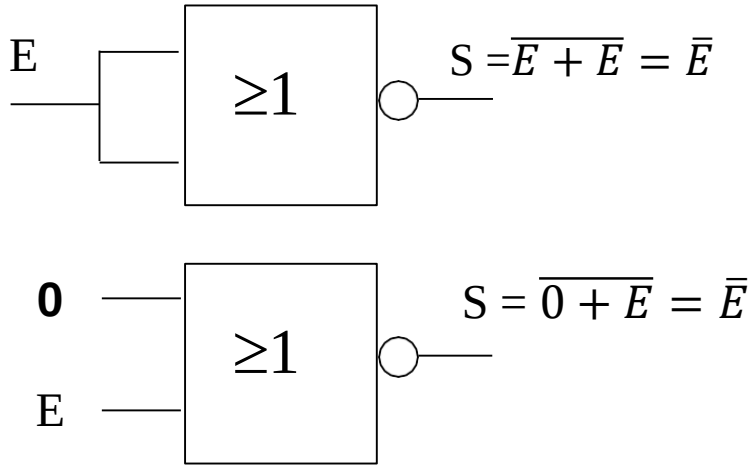
III. Synthèse de Systèmes Combinatoires

A. Les fonctions universelles

A.1) Fonction NON (Inverseur)

E_1	E_2	s
0	0	1
0	1	1
1	0	1
1	1	0





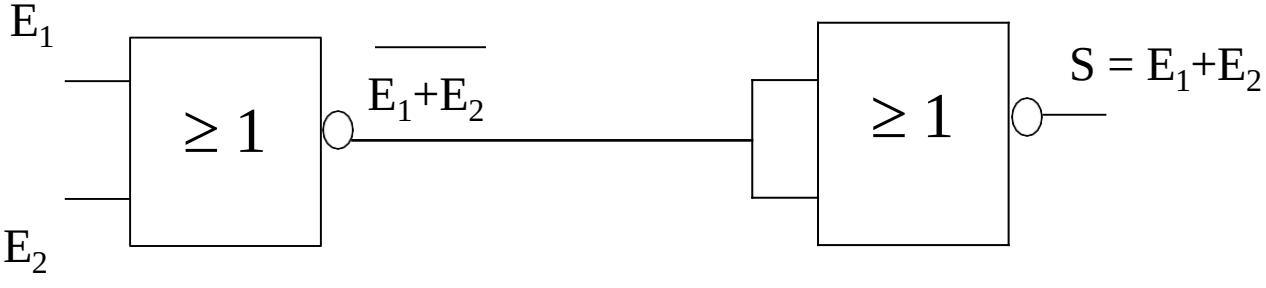
III. Synthèse de Systèmes Combinatoires

A. Les fonctions universelles

A.2) Fonction OU (OR)

Le théorème de De Morgan nous permet d'écrire l'expression booléenne de la fonction OU sous la forme suivante :

$$S = E_1 + E_2 = \overline{\overline{E_1 + E_2}} = \overline{\left(\overline{E_1 + E_2}\right)}$$



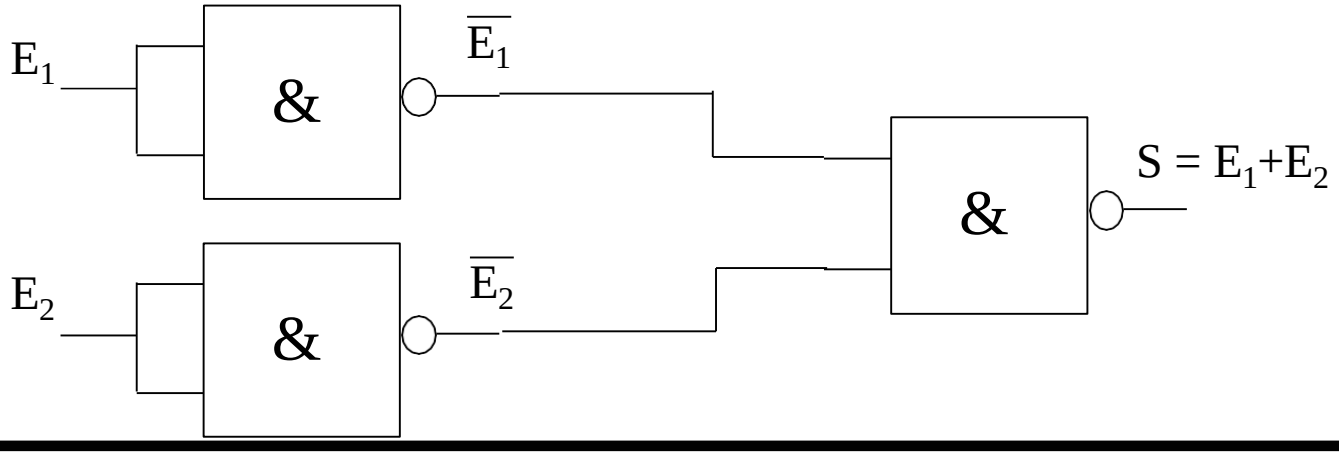
III. Synthèse de Systèmes Combinatoires

A. Les fonctions universelles

A.2) Fonction OU (solution 2)

Mais l'expression ci- dessus peut se transformer en la suivante par l'utilisation de l'un des théorèmes de De Morgan

$$S = E_1 + E_2 = \overline{\overline{E_1 + E_2}} = \overline{\left(\overline{E_1 + E_2}\right)} = \overline{\left(\overline{E_1} \cdot \overline{E_2}\right)}$$

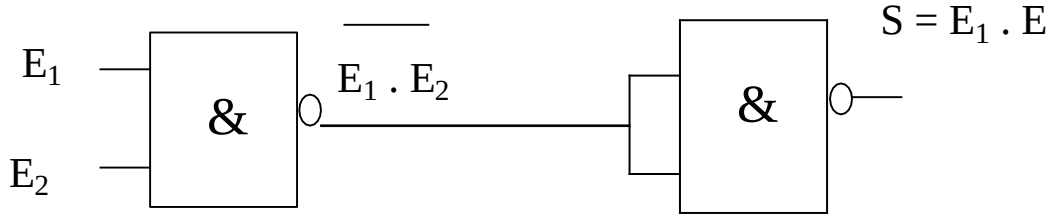


III. Synthèse de Systèmes Combinatoires

A. Les fonctions universelles

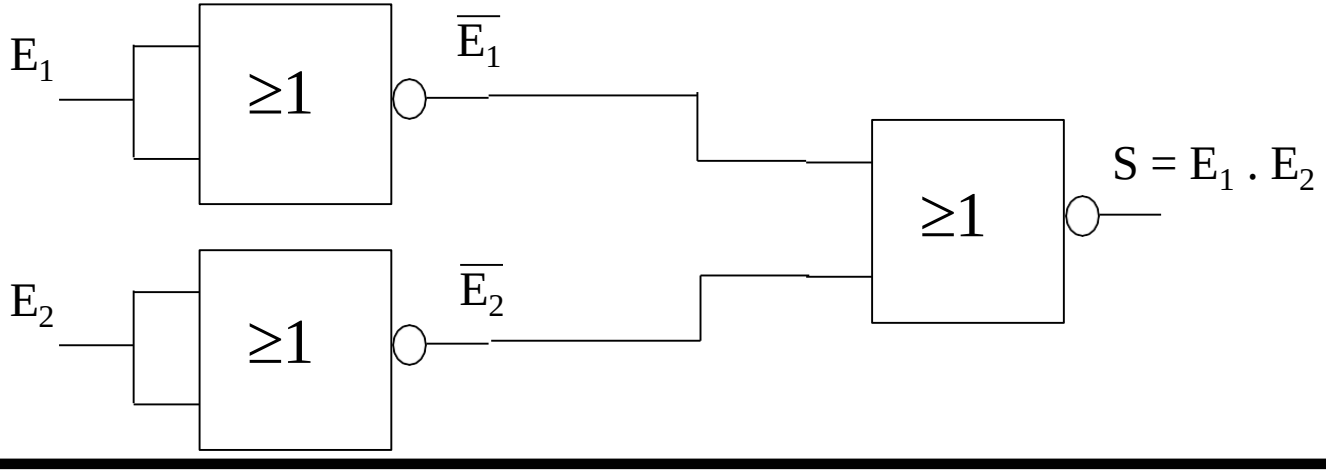
A.3) Fonction ET (AND)

$$S = E_1 \cdot E_2 = \overline{\overline{E_1 \cdot E_2}} = \overline{\left(\overline{E_1 \cdot E_2}\right)}$$



Ou :

$$S = E_1 \cdot E_2 = \overline{\overline{E}_1 \cdot \overline{E}_2} = \overline{\left(\overline{E}_1 \cdot \overline{E}_2\right)} = \overline{\left(\overline{\overline{E}_1 + \overline{E}_2}\right)}$$



III. Synthèse de Systèmes Combinatoires

A. Les fonctions universelles

A.4) Fonction OU EXCLUSIF (XOR)

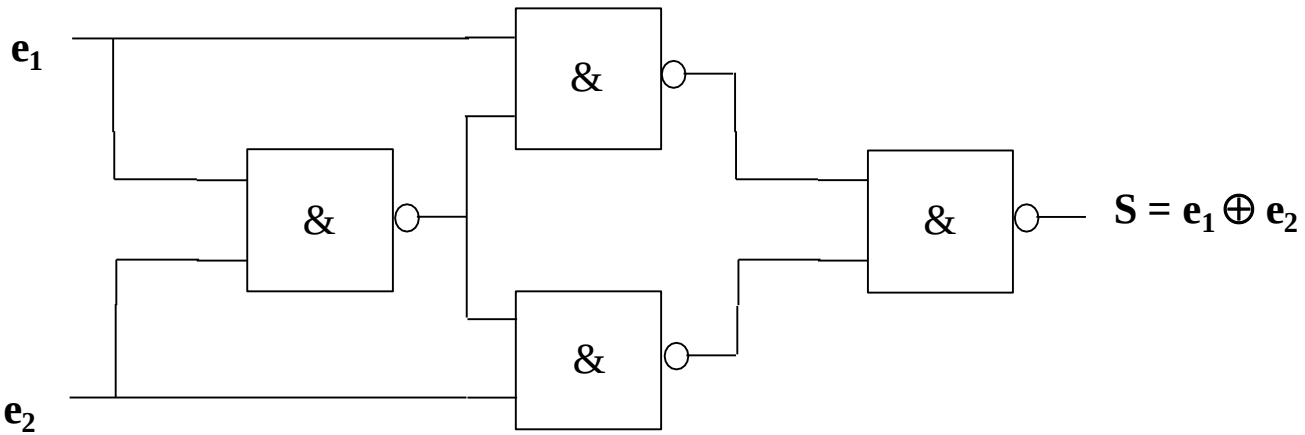
Les formes canoniques nous ont permis d'obtenir l'expression booléenne de la fonction OU EXCLUSIF : $e_1 \oplus e_2 = e_1 \cdot \overline{e_2} + \overline{e_1} \cdot e_2$

En développant :

$$s = e_1 \oplus e_2 = e_1 \cdot \overline{e_2} + \overline{e_1} \cdot e_2 = e_1 \cdot \overline{e_2} + \overline{e_1} \cdot e_2 + e_1 \cdot \overline{e_1} + e_2 \cdot \overline{e_2}$$

$$s = e_1 \cdot \left(\overline{e_1} + \overline{e_2}\right) + e_2 \cdot \left(\overline{e_1} + \overline{e_2}\right) = e_1 \cdot \left(\overline{e_1 \cdot e_2}\right) + e_2 \cdot \left(\overline{e_1 \cdot e_2}\right)$$

$$s = \overline{e_1 \cdot \left(\overline{e_1 \cdot e_2}\right) + e_2 \cdot \left(\overline{e_1 \cdot e_2}\right)} = \overline{e_1 \cdot \left(\overline{e_1 \cdot e_2}\right)} \cdot \overline{e_2 \cdot \left(\overline{e_1 \cdot e_2}\right)}$$



III. Synthèse de Systèmes Combinatoires

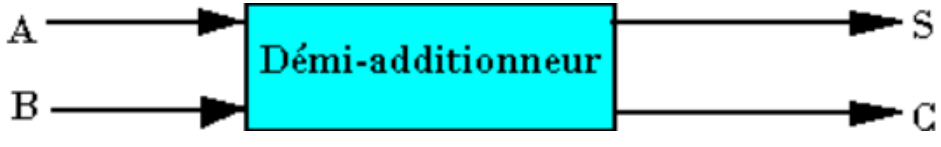
B. Circuits combinatoires

B.1) Demi Additionneur

• Le demi additionneur est un circuit combinatoire qui permet de réaliser la somme arithmétique de deux nombres A et B chacun sur un bit.

- L'addition des nombres binaires se fait en respectant les règles suivantes:
  0 + 0 = 0
  0 + 1 = 1 + 0 = 1
  1 + 1 = 10, On écrit "0" et on reporte "1" sur le bit de rang supérieur (rang de gauche)
  1 + 1 + 1 = 11, on écrit "1" et on reporte "1" sur le bit de rang supérieur

• A la sotie on va avoir la somme S et la retenu C (Carry)



- Pour trouver la structure (le schéma) de ce circuit on doit en premier dresser sa table de vérité

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

1°) Table de vérité:

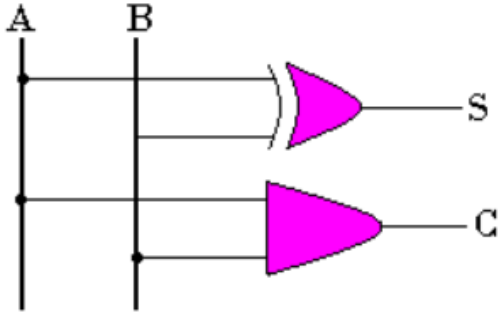
A	b	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



2°) Equation de sortie:

$$S = \overline{A}B + A\overline{B} = A \oplus B$$
$$C = AB$$

3°) Logigramme:



III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

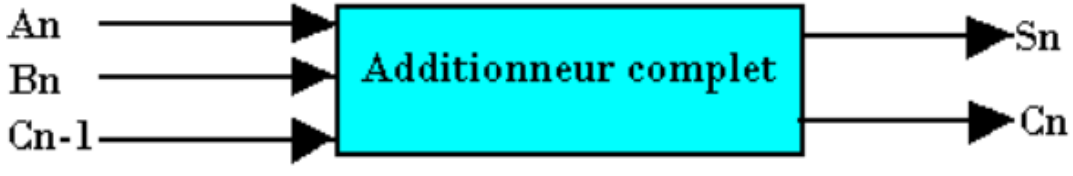
B.2) Additionneur complet

• En binaire lorsque on fait une addition il faut tenir en compte de la retenue entrante.

c_4	c_3	c_2	c_1	$c_0=0$
	a_4	a_3	a_2	a_1
+	b_4	b_3	b_2	b_1
c_4	s_4	s_3	s_2	s_1

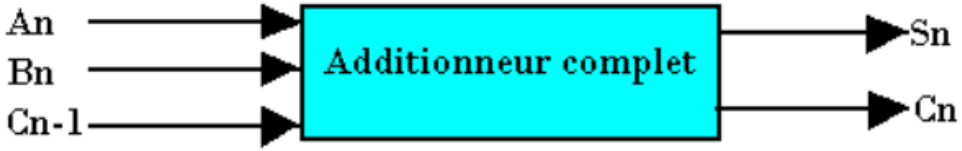
	c_{n-1}
	A_n
+	B_n
c_n	s_n

• Un additionneur complet est un circuit capable de faire la somme de 3 bits A_n et B_n qui sont les bits à additionner et C_{n-1} le retenue vient de l'addition des bits moins significatifs. Le circuit aura deux sorties S_n et C_n qui sont respectivement le bit de la somme et le bit du retenue vient de la somme de 3 bits A_n, B_n et C_{n-1}.



III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires



1°) Table de vérité:

An	Bn	Cn-1	Sn	Cn
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2°) Equation des sorties:

$$Sn = \overline{An}\,\overline{Bn}Cn\text{-}1 + \overline{An}Bn\overline{Cn\text{-}1} + An\overline{Bn}\,\overline{Cn\text{-}1} + AnBnCn\text{-}1$$
$$Cn = \overline{An}BnCn\text{-}1 + An\overline{Bn}Cn\text{-}1 + AnBn\overline{Cn\text{-}1} + AnBnCn\text{-}1$$

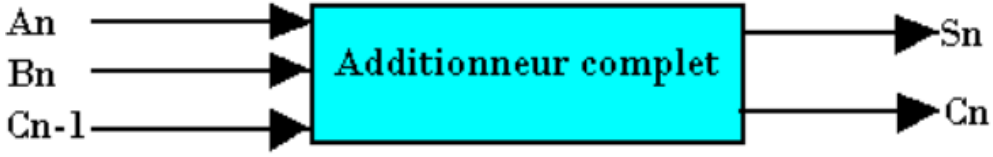
3°) Simplification:

$$S_n = \overline{An}.\overline{Bn}.C_{n-1} + \overline{An}.Bn.\overline{C}_{n-1} + An.\overline{Bn}.\overline{C}_{n-1} + An.Bn.C_{n-1}$$
$$S_n = \overline{An}.(\overline{Bn}.C_{n-1} + Bn.\overline{C}_{n-1}) + An.(\overline{Bn}.\overline{C}_{n-1} + Bn.C_{n-1})$$
$$S_n = \overline{An}(Bn \oplus C_{n-1}) + An\overline{(Bn \oplus C_{n-1})}$$
$$S_n = An \oplus Bn \oplus C_{n-1}$$

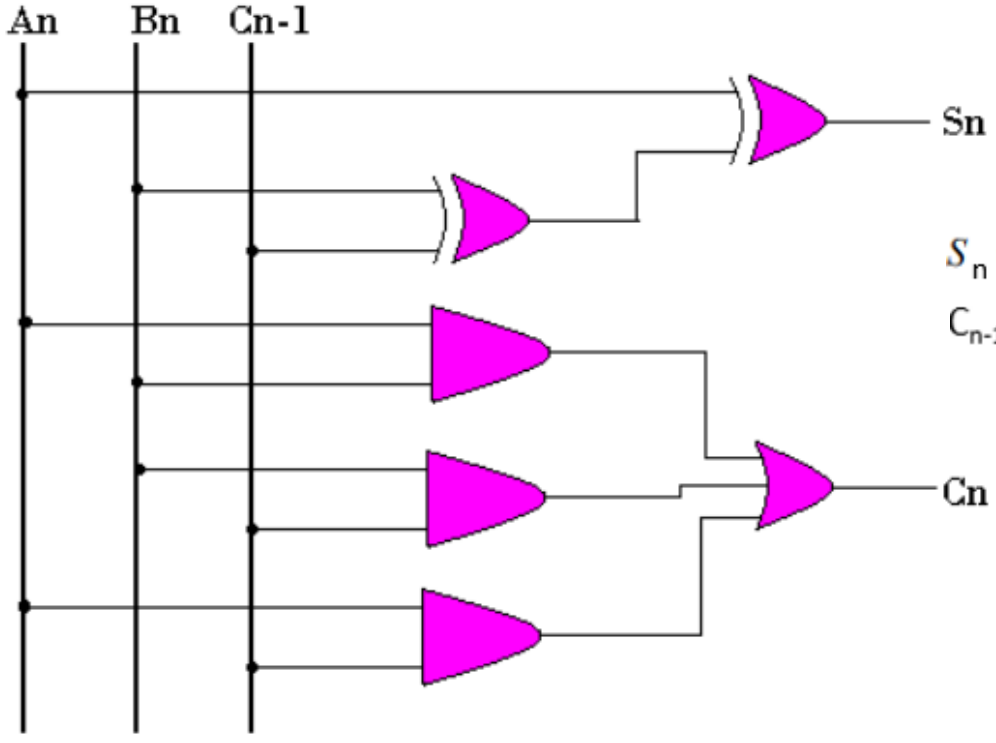
$$C_{n-1} = \overline{An}Bn\,C_{n-1} + An\,\overline{Bn}\,C_{n-1} + An\,Bn\,\overline{C}_{n-1} + An\,BnC_{n-1}$$
$$C_{n-1} = C_{n-1}.(\overline{An}.Bn + An.\overline{Bn}) + AnBn(\overline{C}_{n-1} + C_{n-1})$$
$$C_{n-1} = C_{n-1}.(An \oplus Bn) + AnBn$$

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires



4°) Logigramme:



$$S_n = An \oplus Bn \oplus C_{n-1}$$

$$C_{n-1} = C_{n-1}.(An \oplus Bn) + AnBn$$

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.2) Additionneur complet

B.2.a) Additionneur sur 4 bits

• Un additionneur sur 4 bits est un circuit qui permet de faire l'addition de deux nombres A et B de 4 bits chacun : 1- A(a3a2a1a0) 2- B(b3b2b1b0) 3- La retenu entrante

• En sortie on va avoir le résultat sur 4 bits ainsi que la retenu (5 bits en sortie)

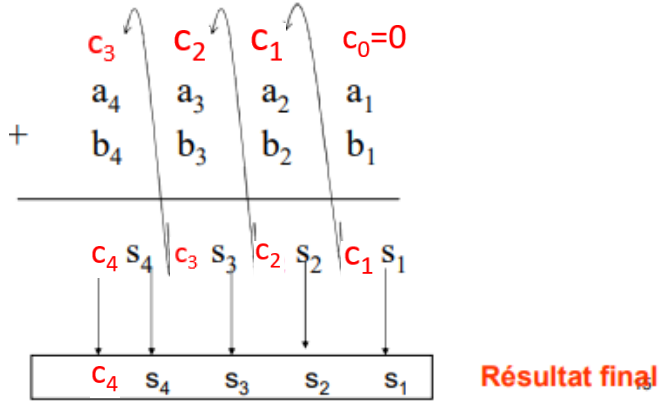
• Donc au total le circuit possède 9 entrées et 5 sorties.

• Avec 9 entrées on a 2^9=512 combinaisons !!!!!! Comment faire pour représenter la table de vérité?

Il faut trouver une solution plus facile et plus efficace pour concevoir ce circuit ?

• Lorsque on fait l'addition en binaire , on additionne bit par bit en commençant à partir du poids fiable et à chaque fois on propage la retenue sortante au bit du rang supérieur.

L'addition sur un bit peut se faire par un additionneur complet sur 1 bits.



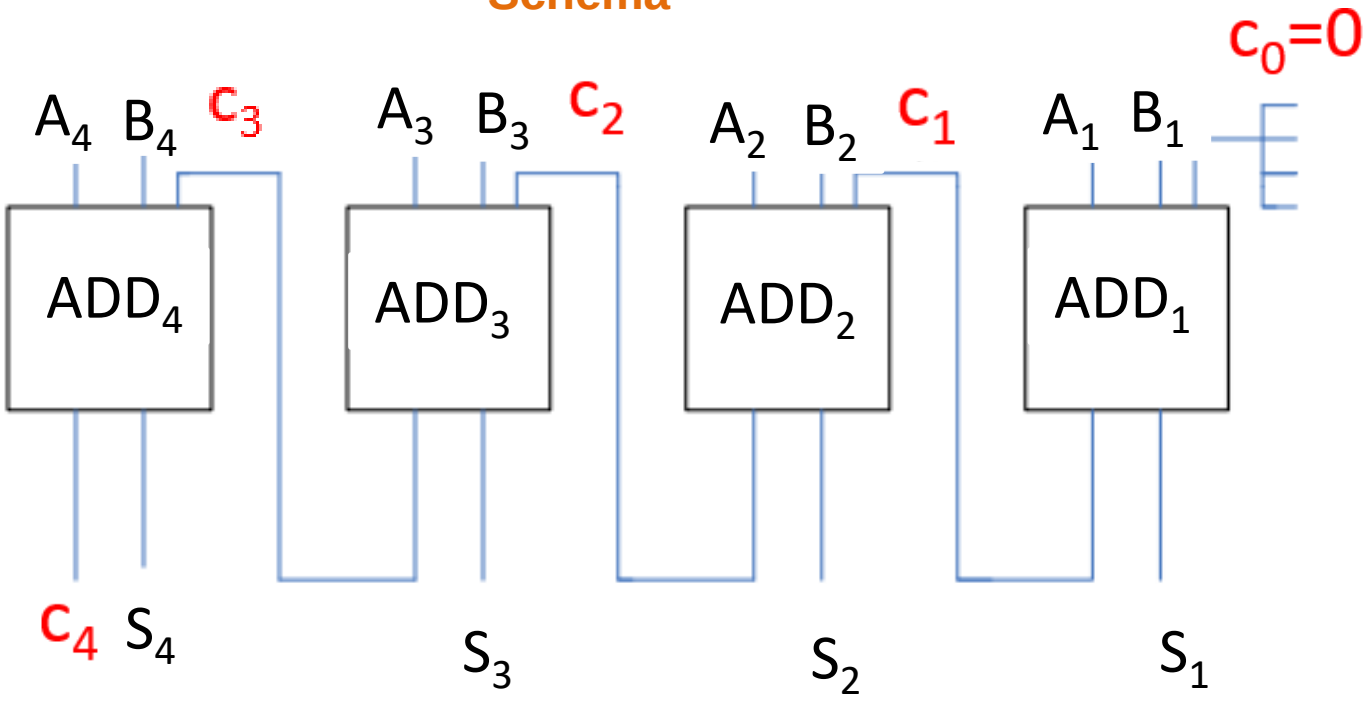
III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.2) Additionneur complet

B.2.a) Additionneur sur 4 bits

Schéma



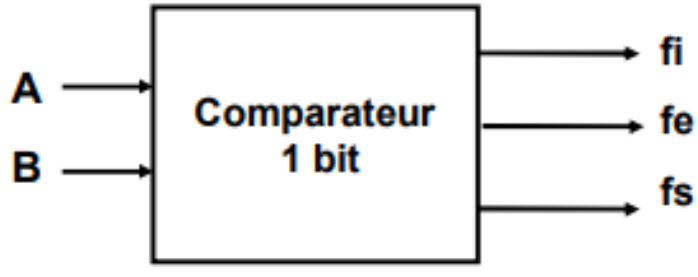
III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.3) Comparateur

- C'est un circuit combinatoire qui permet de comparer entre deux nombres binaire A et B.
- Il possède 2 entrées :
  - A : sur un bit
  - B : sur un bit

- Il possède 3 sorties
  - fe : égalité (A=B)
  - fi : inférieur (A < B)
  - fs : supérieur (A > B)



III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.3) Comparateur

B.3.a) Comparateur sur un bit

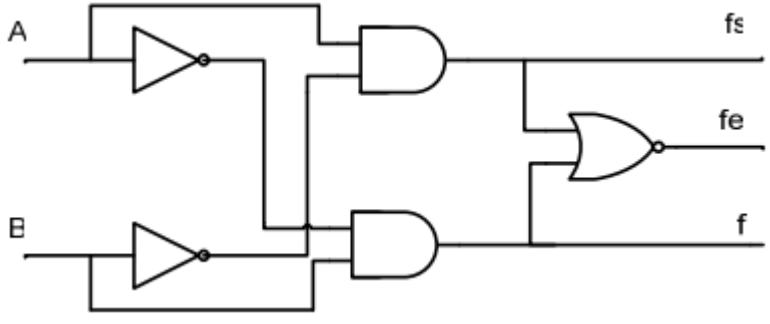
A	B		fs	fe	fi
0	0		0	1	0
0	1		0	0	1
1	0		1	0	0
1	1		0	1	0

$$fs = A.\overline{B}$$

$$fi = \overline{A}B$$

$$fe = \overline{A}\overline{B} + AB = \overline{A \oplus B} = \overline{fs + fi}$$

Schéma :



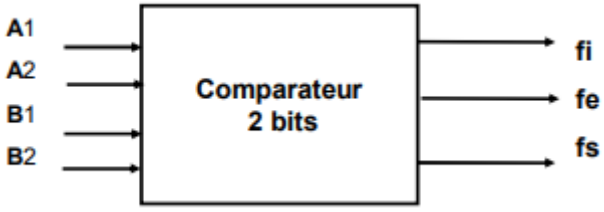
III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.3) Comparateur

B.3.b) Comparateur sur 2 bits

• Il permet de faire la comparaison entre deux nombres A(A2 A1) et B(B2 B1) chacun sur deux bits



1. A=B si

A2=B2 et A1=B1

$$fe = (\overline{A2 \oplus B2}).(\overline{A1 \oplus B1})$$

2. A>B si

A2 > B2 ou (A2=B2 et A1>B1)

$$fs = A2.\overline{B2} + (\overline{A2 \oplus B2}).(A1.\overline{B1})$$

3. A<B si

A2 < B2 ou (A2=B2 et A1<B1)

$$fi = \overline{A2}.B2 + (\overline{A2 \oplus B2}).(\overline{A1}.B1)$$

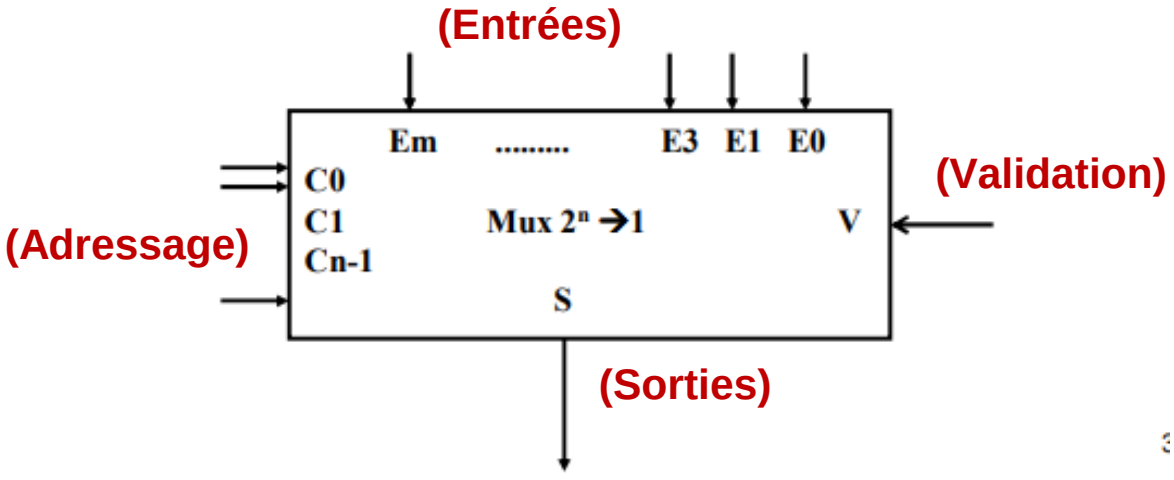
A2	A1	B2	B1	fs	fe	fi
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.4) Multiplexeur

- Un multiplexeur est un circuit combinatoire qui permet de sélectionner une information (1 bit) parmi 2^n valeurs en entrée.
- Il possède :
  - 2^n entrées d'information
  - Une seule sortie
  - N entrées de sélection **Ou de commande (appelées adressage)**



III. Synthèse de Systèmes Combinatoires

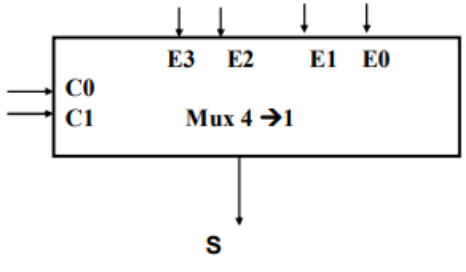
B. Circuits combinatoires

B.4) Multiplexeur

Exemples 1 :

Multiplexeur 4 →1

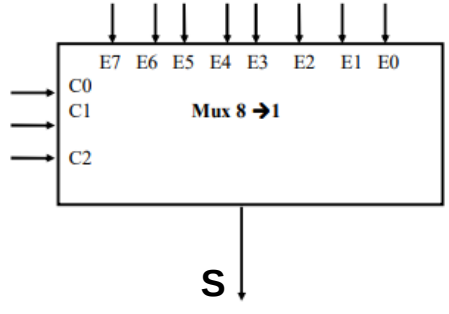
C1	C0		S
0	0		E0
0	1		E1
1	0		E2
1	1		E3



$$S = \overline{C1}.\overline{C0}.(E0) + \overline{C1}.C0.(E1) + C1.\overline{C0}.(E2) + C1.C0.(E3)$$

Multiplexeur 8→1

C2	C1	C0		S
0	0	0		E0
0	0	1		E1
0	1	0		E2
0	1	1		E3
1	0	0		E4
1	0	1		E5
1	1	0		E6
1	1	1		E7



$$S = \overline{C2}.\overline{C1}.\overline{C0}.(E0) + \overline{C2}.\overline{C1}.C0(E1) + \overline{C2}.C1.\overline{C0}(E2) + \overline{C2}.C1.C0(E3) + C2.\overline{C1}.\overline{C0}(E4) + C2.\overline{C1}.C0(E5) + C2.C1.\overline{C0}(E6) + C2.C1.C0(E7)$$

B.4) Multiplexeur

Exemples 2 : Réalisation d'un additionneur complet avec des multiplexeurs 8→1

•Nous avons besoin d'utiliser deux multiplexeurs :Le premier pour réaliser la fonction de la somme et l'autres pour donner la retenue.

a_i	b_i	r_{i-1}		r_i
0	0	0		0
0	0	1		0
0	1	0		0
0	1	1		1
1	0	0		0
1	0	1		1
1	1	0		1
1	1	1		1

a_i	b_i	r_{i-1}		S_i
0	0	0		0
0	0	1		1
0	1	0		1
0	1	1		0
1	0	0		1
1	0	1		0
1	1	0		0
1	1	1		1

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.4) Multiplexeur

Exemples 2 : Réalisation d'un additionneur complet avec des multiplexeurs 8→1

Réalisation de la fonction de la somme

$$S_i = \overline{A_i}.\overline{B_i}.\overline{R_{i-1}}(0) + \overline{A_i}.\overline{B_i}.R_{i-1}(1) + \overline{A_i}.B_i.\overline{R_{i-1}}(1) + \overline{A_i}.B_i.R_{i-1}(0) + A_i.\overline{B_i}.\overline{R_{i-1}}(1) + A_i.\overline{B_i}.R_{i-1}(0) + A_i.B_i.\overline{R_{i-1}}(0) + A_i.B_i.R_{i-1}(1)$$

$$S = \overline{C2}.\overline{C1}.\overline{C0}.(E0) + \overline{C2}.\overline{C1}.C0(E1) + \overline{C2}.C1.\overline{C0}(E2) + \overline{C2}.C1.C0(E3) + C2.\overline{C1}.\overline{C0}(E4) + C2.\overline{C1}.C0(E5) + C2.C1.\overline{C0}(E6) + C2.C1.C0(E7)$$

On pose :

C2=A_i

C1=B_i

C0=R_{i-1}

E0=0, E1=1, E2=1, E3=0, E4=1, E5=0, E6=0, E7=1

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.4) Multiplexeur

Exemples 2 : Réalisation d'un additionneur complet avec des multiplexeurs 8→1

Réalisation de la fonction de la retenue

$$R_i = \overline{A_i}\,\overline{B_i}\,\overline{R_{i-1}}.(0) + \overline{A_i}\,\overline{B_i}R_{i-1}.(0) + \overline{A_i}B_i\overline{R_{i-1}}.(0) + \overline{A_i}B_iR_{i-1}.(1) + A_i\overline{B_i}\,\overline{R_{i-1}}.(0) + A_i\overline{B_i}R_{i-1}.(1)$$
$$+ A_iB_i\overline{R_{i-1}}.(1) + A_iB_iR_{i-1}.(1)$$

$$S = \overline{C2}.\overline{C1}.\overline{C0}.(E0) + \overline{C2}.\overline{C1}.C0(E1) + \overline{C2}.C1.\overline{C0}(E2) + \overline{C2}.C1.C0(E3) +$$
$$C2.\overline{C1}.\overline{C0}(E4) + C2.\overline{C1}.C0(E5) + C2.C1.\overline{C0}(E6) + C2.C1.C0(E7)$$

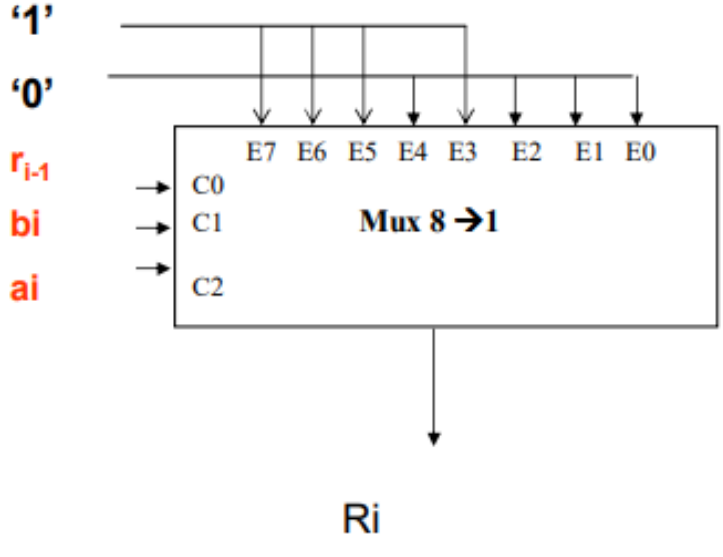
On pose :
C2=A_i
C1=B_i
C0=R_{i-1}
E0=0, E1=0, E2=0, E3=1, E4=0, E5=1, E6=1, E7=1

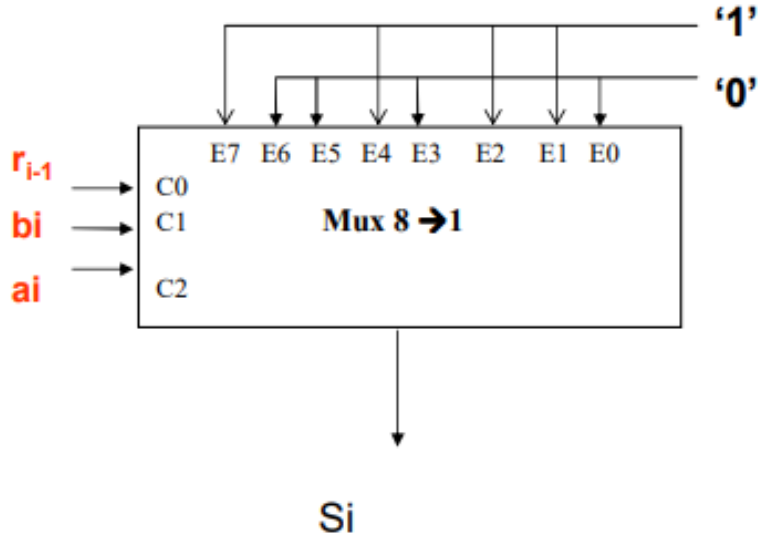
III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.4) Multiplexeur

Exemples 2 : Réalisation d'un additionneur complet avec des multiplexeurs 8→1



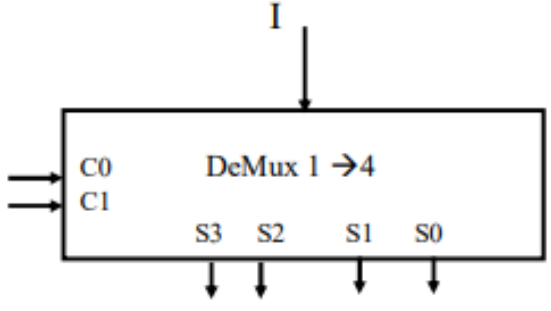


III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.5) Demultiplexeur

- Il joue le rôle inverse d'un multiplexeurs, il permet de faire passer une information dans l'une des sorties selon les valeurs des entrées de commandes.
- Il possède :
  - une seule entrée
  - 2^n sorties
  - N entrées de sélection (commandes)



B. Circuits combinatoires

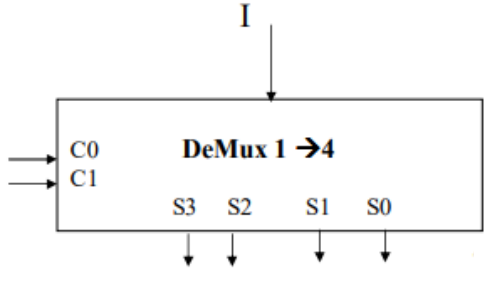
B.5) Demultiplexeur

Exemple :

Demultiplexeur 1→4

C1	C0	S3	S2	S1	S0
0	0	0	0	0	i
0	1	0	0	i	0
1	0	0	i	0	0
1	1	i	0	0	0

$$S0 = \overline{C1}.\overline{C0}.(I)$$
$$S1 = \overline{C1}.C0.(I)$$
$$S2 = C1.\overline{C0}.(I)$$
$$S3 = C1.C0.(I)$$

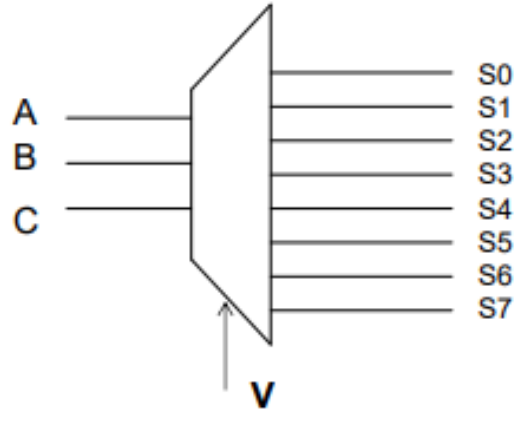


III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.6) Decodeur

- C'est un circuit combinatoire qui est constitué de :
  - N : entrées de données
  - 2^n sorties
  - Pour chaque combinaison en entrée une seule sortie est active à la fois



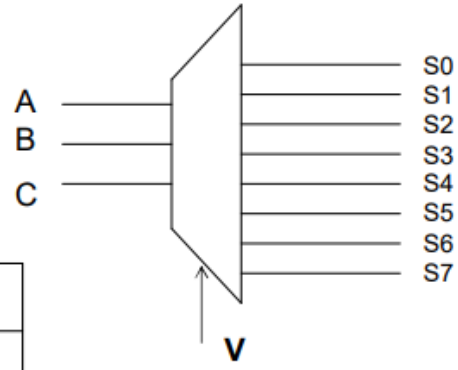
Un décodeur 3→8

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.6) Decodeur

Exemples 1 : **Décodeur 3→8**



A	B	C		S0	S1	S2	S3	S4	S5	S6	S7
0	0	0		1	0	0	0	0	0	0	0
0	0	1		0	1	0	0	0	0	0	0
0	1	0		0	0	1	0	0	0	0	0
0	1	1		0	0	0	1	0	0	0	0
1	0	0		0	0	0	0	1	0	0	0
1	0	1		0	0	0	0	0	1	0	0
1	1	0		0	0	0	0	0	0	1	0
1	1	1		0	0	0	0	0	0	0	1

$$S_0 = \overline{A}.\overline{B}.\overline{C}$$
$$S_1 = \overline{A}.\overline{B}.C$$
$$S_2 = \overline{A}.B.\overline{C}$$
$$S_3 = \overline{A}.B.C$$
$$S_4 = A.\overline{B}.\overline{C}$$
$$S_5 = A.\overline{B}.C$$
$$S_6 = A.B.\overline{C}$$
$$S_7 = A.B.C$$

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.6) Decodeur

Exemples 2 : Réalisation d'un additionneur complet avec des décodeurs binaire 3→8

On a : $S_i = \overline{A}_i.\overline{B}_i.R_{i-1} + \overline{A}_i.B_i.\overline{R}_{i-1} + A_i.\overline{B}_i.\overline{R}_{i-1} + A_i.B_i.R_{i-1}$

0 0 1 **0 1 0** **1 0 0** **1 1 1**

$R_i = \overline{A}_i B_i R_{i-1} + A_i \overline{B}_i R_{i-1} + A_i B_i \overline{R}_{i-1}. + A_i B_i R_{i-1}$

0 1 1 **1 0 1** **1 1 0** **1 1 1**

On pose $A=A_i$, $B=B_i$, $C=R_{i-1}$

$$S_0 = \overline{A}.\overline{B}.\overline{C}, S_1 = \overline{A}.\overline{B}.C, S_2 = \overline{A}.B.\overline{C}, S_3 = \overline{A}.B.C,$$

$$S_4 = A.\overline{B}.\overline{C}, S_5 = A.\overline{B}.C, S_6 = A.B.\overline{C}, S_7 = A.B.C$$

Donc : $R_i = S3 + S5 + S6 + S7$

$S_i = S1 + S2 + S4 + S7$

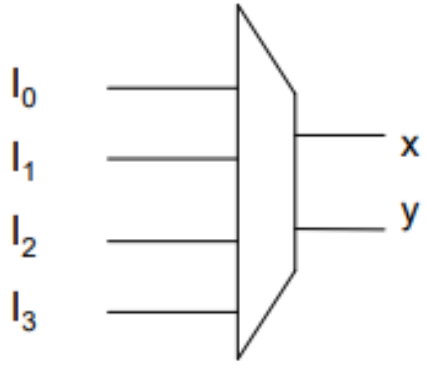
III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.7) Encodeur

- Il joue le rôle inverse d'un décodeur
  - Il possède 2^n entrées
  - N sortie
  - Pour chaque combinaison en entrée on va avoir sont numéro (en binaire) à la sortie.

Encodeur 4→2



III. Synthèse de Systèmes Combinatoires

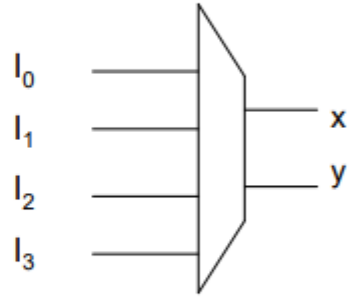
B. Circuits combinatoires

B.7) Encodeur

Exemples 1 :

L'encodeur binaire (4→2)

I_0	I_1	I_2	I_3	x	y
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1



$$X = I2 + I3$$

$$Y = I1 + I3$$

Noter :

Cas 1 : Encodeur simple (non prioritaire)

⚠ Règle : on n'autorise qu'une seule entrée = 1 à la fois (les cas 011, 101, 110, 111 sont interdits)

Table (seulement les cas autorisés) :

E2	E1	E0	B	A	Commentaire
0	0	1	0	0	encode E0 → code 0
0	1	0	0	1	encode E1 → code 1
1	0	0	1	0	encode E2 → code 2

Équations (valables seulement si une seule entrée vaut 1) :

A = E1 (simple)
B = E2 (simple)

Cas 2 : Encodeur prioritaire (E2 > E1 > E0) - table complète

⚠ Règle : on AUTORISE plusieurs entrées = 1 d'après l'exercice (la priorité décide la sortie, par exemple ici E2 prioritaire sur E1, E1 prioritaire sur E0)

Table complète (8 lignes possibles)

E2	E1	E0	B	A	Explication
0	0	1	0	0	seule E0 active → code 0
0	1	0	0	1	seule E1 active → code 1
0	1	1	0	1	E1 prioritaire sur E0 → code 1
1	0	0	1	0	seule E2 active → code 2
1	0	1	1	0	E2 prioritaire sur E0 → code 2
1	1	0	1	0	E2 prioritaire sur E1 → code 2
1	1	1	1	0	E2 prioritaire sur E1 et E0 → code 2

B = E2 (dès que E2 = 1, B = 1, on se fiche des autres)

$A = \overline{E2}\,E1$ (A ne vaut 1 que si E1 = 1 **et** E2 = 0, peu importe E0, donc il faut inverser E2

Cas 3 : Encodeur prioritaire (table condensée avec X)

⚠ Ici on garde seulement les lignes utiles, en utilisant X = "on se fiche de cette valeur":

E2	E1	E0	B	A
0	0	0	0	0
0	0	1	0	0
0	1	X	0	1
1	X	X	1	0

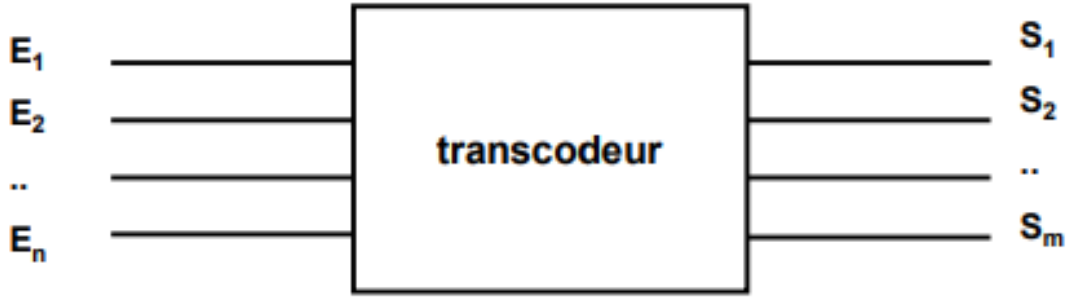
→ On a éliminé 4 lignes car la sortie est la même.
→ Donc les équations : A = E1 et B = E2

III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.8) Transcodeur

C'est un circuit combinatoire qui permet de transformer un code X (sur n bits) en entrée en un code Y (sur m bits) en sortie.



III. Synthèse de Systèmes Combinatoires

B. Circuits combinatoires

B.8) Transcodeur

Exemple : Transcodeur BCD/EXESS3

A	B	C	D	X	Y	Z	T
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

Électronique séquentiel